LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

12

Linux Logging, Monitoring and Scheduling

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 Introduction
02 Understanding Linux Logs
03 systemd Journal: journalctl
04 Log Rotation: logrotate
05 Server Health Monitoring
06 Monitoring Processes
07 Scheduling with cron
08 Cron Special Strings
09 One-Time Tasks: at
10 systemd Timers
11 Monitoring Tools
12 Troubleshooting Scenarios
13 Cloud and DevOps Monitoring
14 Best Practices
15 Interview Q&A
16 Takeaways and Exercises

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

Part 1 · Core Linux

github.com/rv-ops/linux-interview-preparation-guide

CHAPTER OBJECTIVE

Understand where Linux stores logs and how to read them, how to monitor CPU, memory, disk, I/O and network health, and how to schedule tasks using cron, `at` and systemd timers. Learn how these skills are used to troubleshoot production issues and how centralized monitoring platforms fit into DevOps environments.

1 Introduction

In production, servers and applications continuously generate information about their activity. DevOps engineers use it to understand system health, find failures, investigate incidents and automate routine work.

Three important areas of Linux operations are:

- **Logging:** Recording system and application events.
- **Monitoring:** Observing server health, resource usage and service availability.
- **Scheduling:** Running commands and scripts automatically at specific times or intervals.

For example, when a website becomes unavailable, a DevOps engineer may need to:

1. Check whether the application service is running.
2. Inspect system and application logs.
3. Check CPU, memory and disk usage.
4. Identify when the issue started.
5. Schedule recurring health checks or maintenance tasks.

2 Understanding Linux Logs

Linux logs record events generated by the operating system, applications, services and security components. They help answer questions such as:

- When did a service fail?
- Why did an application crash?
- Was someone attempting an unauthorized login?
- Did the server run out of disk space?
- What happened during the last reboot?

Common Log Locations

Most Linux distributions store log files under `/var/log/`.

Log file	Purpose
`/var/log/syslog`	General system messages (Ubuntu/Debian)
`/var/log/messages`	General system messages (RHEL-based)
`/var/log/auth.log`	Authentication and sudo events (Ubuntu/Debian)
`/var/log/secure`	Authentication and security events (RHEL-based)
`/var/log/kern.log`	Kernel messages, where maintained
`/var/log/dmesg`	Boot and kernel messages, where maintained
`/var/log/nginx/`	Nginx access and error logs
`/var/log/apache2/` / `/var/log/httpd/`	Apache logs (Ubuntu / RHEL-based)

Log locations vary depending on the distribution and logging configuration.

> **INTERVIEW NOTE**
>
> On most modern systems, **systemd-journald** collects logs first. **rsyslog** then writes them to text files such as `/var/log/syslog` or `/var/log/messages`. Some minimal or container images have no rsyslog, so only `journalctl` shows the logs.

Viewing Log Files

```
# List log files and sizes
ls -lh /var/log/

# Read a large log safely (page by page)
less /var/log/syslog

# Last 50 lines
tail -n 50 /var/log/syslog

# Follow new entries in real time
tail -f /var/log/syslog

# Search for errors
grep -i "error" /var/log/syslog
```

DEVOPS EXAMPLE

If a deployment fails, check the application log and system logs to find whether the problem is configuration, permissions, dependencies or system resources.

3 systemd Journal and journalctl

Modern distributions use systemd to manage services and collect their logs in the systemd journal. `journalctl` queries these logs. Basic usage was introduced in Chapter 5; this section is a quick reference.

```
# Latest entries, and follow in real time
journalctl -n 50
journalctl -f

# Current boot and previous boot (if retained)
journalctl -b
journalctl -b -1

# A specific service
journalctl -u nginx -n 100
journalctl -u nginx -f

# Time filters
journalctl --since "1 hour ago"
journalctl --since "2026-11-05 09:00:00" --until "2026-11-05 10:00:00"

# Only errors and more severe messages
journalctl -p err
```

Log Priorities

Priority	Name	Meaning
0	emerg	System is unusable
1	alert	Action must be taken immediately
2	crit	Critical condition
3	err	Error
4	warning	Warning
5	notice	Normal but significant
6	info	Informational
7	debug	Debug messages

`journalctl -p warning` shows warnings and everything more severe (priorities 0 to 4).

Journal Disk Usage

```
# How much space the journal uses
journalctl --disk-usage

# Remove entries older than 7 days
sudo journalctl --vacuum-time=7d
```

Logs from previous boots exist only if the journal is persistent (see Chapter 5).

> **IMPORTANT**
> Do not delete production logs casually. They may be needed for incident investigation, auditing and compliance.

4 Log Rotation – logrotate

Log files grow continuously. For example, an Nginx access log grows with every request. Without management, logs can fill the disk. Linux commonly uses `logrotate` to:

- Rotate old log files.
- Compress previous logs.
- Remove logs after a retention period.

- Create new log files.
- Notify services after rotation, when configured.

logrotate usually runs once a day, triggered by cron (`/etc/cron.daily/logrotate`) or by a systemd timer (`logrotate.timer`).

Configuration Files

```
/etc/logrotate.conf      → Main configuration
/etc/logrotate.d/        → Application-specific configurations
```

Example: Nginx

```
/var/log/nginx/*.log {
    daily
    rotate 7
    missingok
    notifempty
    compress
    delaycompress
    sharedscripts
    postrotate
        systemctl reload nginx
    endscript
}
```

Directive	Purpose
`daily`	Rotate logs daily
`rotate 7`	Keep seven rotated log files
`missingok`	Do not report an error if the log file is missing
`notifempty`	Do not rotate empty log files
`compress`	Compress rotated logs
`delaycompress`	Compress on the next rotation, not immediately
`sharedscripts`	Run the script once for all matching logs
`postrotate`	Run commands after rotation (here, tell Nginx to reopen its logs)
`copytruncate`	Copy the log, then empty the original; for apps that cannot reopen log files

INTERVIEW TRAP

After rotation, an application keeps writing to the old (renamed) file unless it is told to reopen its logs. That is why `postrotate` reloads the service. If the application cannot reopen logs, use `copytruncate`, but a few lines may be lost during the copy.

Test logrotate

```
# Debug mode: shows what would happen, makes no changes
sudo logrotate -d /etc/logrotate.conf

# Force a rotation now
sudo logrotate -f /etc/logrotate.conf
```

Use forced rotation carefully on production systems.

INTERVIEW LINE

Log rotation prevents log files from growing indefinitely and helps control disk usage.

5 Server Health Monitoring

Monitoring helps find performance problems before they cause downtime. Key areas are CPU, memory, disk usage, disk I/O, load average, network activity, running processes and service availability.

CPU

```
# Live process and resource view
top

# Interactive alternative (may need installing)
htop

# CPU details
lscpu

# Per-CPU statistics every second (sysstat package)
mpstat -P ALL 1
```

Memory

```
free -h
```

```
               total        used        free      shared  buff/cache   available
Mem:           7.7Gi       3.2Gi       1.0Gi       200Mi       3.5Gi       4.1Gi
Swap:          2.0Gi       100Mi       1.9Gi
```

- `total` : Total installed memory.
- `used` : Memory in use by processes and the kernel.
- `free` : Completely unused memory.
- `buff/cache` : Memory used for buffers and filesystem cache.
- `available` : Estimated memory available for new applications without swapping.

> **IMPORTANT**
>
> Linux uses unused memory for caching. Low `free` memory does not mean the server is running out of memory. Check `available` and swap activity instead.

Disk Usage

```
# Filesystem usage
df -h

# Size of a directory
du -sh /var/log/

# Largest directories under /var
sudo du -h --max-depth=1 /var | sort -hr

# Large log files
sudo find /var/log -type f -size +100M
```

Load Average

```
uptime
```

```
10:30:00 up 15 days,  2 users,  load average: 2.10, 1.80, 1.50
```

The three values are the averages over the last 1, 5 and 15 minutes. Load average counts tasks that are running, waiting for CPU, or waiting in uninterruptible sleep (usually disk I/O). It is not a CPU

percentage.

Compare it with the number of CPUs (`nproc`). A load of 4 on a 4-vCPU server means it is fully busy; the same load on a single-CPU server means tasks are waiting.

Disk I/O

```
# Extended statistics every second
iostat -xz 1
```

Field	Meaning
`r/s` , `w/s`	Read and write operations per second
`r_await` , `w_await`	Average time (ms) for read and write requests, including queue time
`%util`	How busy the device is

High I/O wait time can make applications slow even when CPU usage is low. Older versions of iostat show a single `await` column.

Network

```
# Interfaces and IP addresses
ip addr

# Listening ports
ss -tulnp

# Connection summary
ss -s

# Interface statistics, including errors and drops
ip -s link

# Live traffic per connection (may need installing)
sudo iftop
```

6 Monitoring Running Processes

A process consuming excessive CPU or memory can slow down the whole server.

```
# All processes
ps aux

# Top CPU consumers
ps aux --sort=-%cpu | head

# Top memory consumers
ps aux --sort=-%mem | head

# Details of one process (replace 1234 with the real PID)
ps -p 1234 -o pid,ppid,%cpu,%mem,cmd
```

Key in top	Purpose
`P`	Sort by CPU usage
`M`	Sort by memory usage
`1`	Show each CPU separately
`k`	Send a signal to a process
`q`	Quit

PRODUCTION EXAMPLE

If an application responds slowly, check whether a process is using unusually high CPU or memory before restarting anything.

7 Scheduling Tasks with cron

`cron` runs commands and scripts automatically at scheduled times. Common uses include daily backups, cleaning temporary files, health checks, report generation and file synchronization. Chapter 5 covered cron in detail; this is a quick recap.

```
# Service status (Ubuntu: cron, RHEL-based: crond)
systemctl status cron
systemctl status crond

# Edit and list the current user's jobs
crontab -e
crontab -l
```

WARNING

`crontab -r` removes **all** of the current user's cron jobs without confirmation. Back up first: `crontab -l > crontab-backup.txt`.

Cron Syntax

```
* * * * * command
| | | | |
| | | | +--- Day of week (0-7, Sunday is 0 or 7)
| | | +----- Month (1-12)
| | +------- Day of month (1-31)
| +--------- Hour (0-23)
+----------- Minute (0-59)
```

Schedule	Expression
Every minute	`* * * * *`
Every five minutes	`*/5 * * * *`
Every day at 2 AM	`0 2 * * *`
Every Sunday at 3 AM	`0 3 * * 0`
Monday to Friday at 9 AM	`0 9 * * 1-5`
First day of every month at midnight	`0 0 1 * *`

Redirect Cron Output

Cron jobs run without a terminal. Redirect output so failures can be investigated:

```
0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1
```

Important Cron Considerations

1. Use absolute paths for scripts and commands.
2. Make sure the script has execute permission.
3. Set required environment variables inside the script; cron uses a minimal environment.
4. Redirect output to a log file.
5. Test the script manually before scheduling it.
6. Prevent overlapping runs with `flock` if a job can take longer than its interval (see Chapter 10).

```
#!/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

/usr/bin/df -h >> /var/log/disk-report.log 2>&1
```

8 Cron Special Strings

Expression	Meaning
`@reboot`	Run once when the cron service starts after boot
`@hourly`	Once every hour
`@daily`	Once a day (midnight)
`@weekly`	Once a week
`@monthly`	Once a month
`@yearly`	Once a year

```
@reboot /opt/scripts/startup-check.sh
```

9 at – Schedule a One-Time Task

Cron is for recurring jobs. `at` runs a command **once** at a specified time.

```
# Install and enable if required
sudo apt install at            # Ubuntu / Debian
sudo dnf install at            # RHEL-based
sudo systemctl enable --now atd

# Schedule a command for 11 PM today
echo "tar -czf /tmp/log-backup.tar.gz /var/log/" | at 23:00

# Other time formats
echo "/opt/scripts/cleanup.sh" | at now + 2 hours

# List pending jobs
atq

# Remove job number 2
atrm 2
```

You can also run `at 23:00`, type commands at the `at>` prompt, and press `Ctrl + D` to submit.

> **DEVOPS USE CASE**
>
> Schedule a one-time maintenance or cleanup task without creating a recurring cron job.

10 systemd Timers – An Alternative to Cron

A systemd timer needs two files: a **service** that defines what to run and a **timer** that defines when. A complete daily backup example was shown in Chapter 5:

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Run Backup Daily

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer

# List timers with their next and last run
systemctl list-timers

# Logs of the job
journalctl -u backup.service
```

By default, a timer starts the service with the same name (`backup.timer` starts `backup.service`). `Persistent=true` runs a missed job after the server comes back up.

Cron vs systemd Timers

Feature	Cron	systemd Timer
Configuration	One crontab line	Service and timer unit files
Logging	Needs output redirection	Automatic in the journal
Missed runs	Skipped if the server was down	Can catch up with `Persistent=true`
Dependencies and limits	Limited	Full systemd features (dependencies, resource limits)
Complexity	Simple	More configuration

For simple recurring tasks, cron is often enough. For jobs that need journal logging, dependencies or missed-run handling, timers are a better fit.

11 Monitoring Tools Commonly Used in Linux

Tool	Purpose
`top` / `htop`	Live process and resource monitoring
`free`	Memory usage
`df` / `du`	Filesystem usage / directory sizes
`iostat`	Disk I/O statistics
`vmstat`	Processes, memory, swap, CPU and I/O
`mpstat`	CPU statistics per core
`sar`	Historical performance statistics
`ss`	Network sockets and connections
`iftop` / `nload`	Network traffic per connection / per interface
`dmesg`	Kernel ring-buffer messages

```
# Install missing tools (sysstat provides iostat, mpstat and sar)
sudo apt install sysstat htop       # Ubuntu / Debian
sudo dnf install sysstat htop       # RHEL-based
```

vmstat

```
vmstat 1
```

Field	Meaning
`r`	Processes waiting to run
`b`	Processes blocked waiting for I/O
`si` / `so`	Memory swapped in / out per second (non-zero values mean active swapping)
`us` / `sy`	CPU time in user space / kernel space
`id`	CPU idle time
`wa`	CPU time waiting for I/O

sar

```
# CPU usage every second, 5 samples
sar -u 1 5

# Historical CPU usage from today's collected data
sar -u
```

Historical reports require data collection to be enabled. On Ubuntu, set `ENABLED="true"` in `/etc/default/sysstat`; on both families, run `sudo systemctl enable --now sysstat`.

> **DEVOPS USE CASE**
>
> Historical data shows whether a performance problem is new or happens regularly, for example every night during backups.

12 Production Troubleshooting Scenarios

Scenario 1 Application service suddenly stopped

```
# Status and recent logs
systemctl status nginx
journalctl -u nginx -n 100

# Is the port listening?
sudo ss -tulnp

# Check configuration syntax
sudo nginx -t

# Restart only when the cause is understood, then verify
sudo systemctl restart nginx
systemctl status nginx
```

Scenario 2 Server disk usage reached 100%

```
df -h
sudo du -h --max-depth=1 /var | sort -hr
sudo find /var/log -type f -size +100M
cat /etc/logrotate.conf
```

Do not delete large files immediately. Check whether they are active logs, application data or old backups. If `df` still shows the disk as full after deleting, a process may be holding a deleted file open (see Chapter 6).

Scenario 3 Server has high CPU usage

```
uptime
nproc
ps aux --sort=-%cpu | head -10
top
```

Then check application logs and recent deployments to find the cause.

Scenario 4 Server is running out of memory

```
# Available memory and swap
free -h
vmstat 1 5

# Top memory consumers
ps aux --sort=-%mem | head -10

# Was any process killed by the OOM killer?
journalctl -k | grep -i "out of memory"
```

Under severe memory pressure, the kernel's OOM killer terminates processes to keep the system running.

Scenario 5 A scheduled backup did not run

```
# Is the job scheduled?
crontab -l

# Is cron running and did it start the job?
systemctl status cron                  # RHEL: crond
journalctl -u cron --since today       # RHEL: crond

# Run the script manually with tracing
bash -x /opt/scripts/backup.sh
```

Verify permissions, absolute paths, environment variables and output redirection.

Scenario 6 Website is slow but CPU usage is low

```
# Memory and swapping
free -h

# I/O wait (wa column) and blocked processes (b column)
vmstat 1 5

# Disk latency
iostat -xz 1

# Connection summary
ss -s

# Web server logs
journalctl -u nginx -n 100
```

Low CPU usage does not mean the server is healthy. The application may be waiting for disk I/O, a database, an external API or the network.

13 Logging and Monitoring in Cloud and DevOps Environments

In production, teams combine local Linux tools with centralized monitoring and logging platforms.

AWS Monitoring

Service	Purpose
Amazon CloudWatch	Metrics, logs, alarms and notifications for AWS resources
CloudWatch Agent	Sends OS-level metrics (memory, disk) and log files from servers to CloudWatch
AWS CloudTrail	Records AWS account activity and API calls: who did what, and when
AWS Systems Manager	Manages and automates operations on EC2 instances

INTERVIEW TRAP

By default, CloudWatch shows EC2 CPU, network and disk I/O metrics, but **not memory or disk-space usage**. These are visible only inside the operating system, so you must install the **CloudWatch Agent** to collect them.

Prometheus and Grafana

- **Prometheus** collects metrics from targets (for example, Node Exporter on Linux servers), stores them as time-series data and evaluates alert rules.
- **Grafana** visualizes metrics in dashboards and connects to Prometheus and other data sources.

ELK / Elastic Stack

- **Elasticsearch:** Stores and searches log data.
- **Logstash:** Processes and transforms data.
- **Kibana:** Visualizes and analyzes data.

Together, they centralize logs from many servers in one place.

Example Monitoring Architecture



INTERVIEW LINE

Linux commands help with immediate troubleshooting on one server; centralized monitoring lets teams observe many servers and applications from one place.

14 Best Practices

1. **Monitor critical resources:** CPU, memory, disk, network and application health.

2. **Centralize logs** from multiple servers in production.
3. **Configure log rotation** to prevent uncontrolled log growth.
4. **Retain important logs** according to retention and compliance requirements.
5. **Schedule backups and test restores**, not just backup creation.
6. **Capture cron output** by redirecting stdout and stderr to log files.
7. **Alert on meaningful conditions**, not every small fluctuation.
8. **Compare with historical metrics** to understand what is normal.
9. **Investigate before restarting:** evidence in memory and logs can disappear after a restart.
10. **Document recurring incidents**, including root causes and fixes.

15 Common Interview Questions

Q1. Where are Linux log files generally stored?

Under `/var/log/`, including system logs, authentication logs and application-specific logs.

Q2. What is journalctl?

`journalctl` views and filters logs collected by the systemd journal.

Q3. How do you check logs for a specific service?

```
journalctl -u nginx
```

Q4. How do you monitor logs in real time?

```
tail -f application.log
journalctl -u nginx -f
```

Q5. What is logrotate?

logrotate manages log files by rotating, compressing and removing old logs according to configured retention rules.

Q6. Why does an application keep writing to the old log file after rotation?

It still has the old file open. Reload the service in `postrotate` so it reopens its logs, or use `copytruncate` for applications that cannot reopen log files.

Q7. What is the difference between top and htop?

Both monitor processes and resource usage. `htop` is more interactive and user-friendly but may need to be installed.

Q8. How do you check memory and disk usage?

`free -h` for memory, `df -h` for filesystems and `du -sh /path` for directory sizes.

Q9. What is load average?

The average number of tasks running, waiting for CPU or waiting in uninterruptible sleep over the last 1, 5 and 15 minutes. Compare it with the number of CPU cores.

Q10. What is cron?

Cron is a scheduling service that runs commands or scripts automatically at recurring intervals.

Q11. What is the difference between cron and at?

Cron runs recurring jobs. `at` runs a job once at a specified time.

Q12. What is a systemd timer?

A systemd timer runs a systemd service at specified times. It provides journal logging and can catch up on missed runs with `Persistent=true`.

Q13. How do you check whether a cron job ran?

Check the cron service logs with `journalctl -u cron` (`crond` on RHEL-based systems) and the output log configured for the job.

Q14. How do you find the process consuming the most CPU?

```
ps aux --sort=-%cpu | head
```

Q15. What is the difference between CloudWatch and CloudTrail?

CloudWatch monitors metrics, logs and alarms. CloudTrail records AWS account activity and API calls for auditing and investigation.

Q16. Why can't you see EC2 memory usage in CloudWatch by default?

Memory and disk-space usage are visible only inside the operating system. Install the CloudWatch Agent on the instance to send these metrics.

16 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ Where Linux stores system and application logs.
- ✓ How to inspect logs using `tail`, `grep` and `journalctl`.
- ✓ How to monitor service logs in real time.
- ✓ How logrotate manages log files.
- ✓ How to monitor CPU, memory, disk, I/O and network activity.
- ✓ How to use `top`, `free`, `df`, `iostat`, `vmstat` and `sar`.
- ✓ How to schedule recurring jobs using cron and one-time tasks using `at`.
- ✓ How systemd timers compare with cron.
- ✓ How to troubleshoot common production issues.
- ✓ How centralized logging and monitoring platforms work in DevOps.

PRACTICAL EXERCISES

Perform these exercises on a Linux test machine.

1. Display the last 100 lines of the system log.
2. Search the authentication log for failed login attempts.
3. Display Nginx logs in real time.
4. Check the logs of a failed systemd service.
5. Check how much disk space `/var/log` uses.
6. Find the top five processes by CPU and by memory usage.
7. Check memory and swap usage using `free -h`.
8. Display disk I/O statistics using `iostat -xz 1`.
9. Create a cron job that runs a disk usage check every five minutes.
10. Create a daily backup cron job that saves its output to a log file.
11. Schedule a one-time task using `at` and confirm it with `atq`.
12. Create a simple systemd timer that runs a script daily.
13. Create a test logrotate rule and validate it in debug mode.
14. Investigate a simulated application failure using `systemctl status` and `journalctl`.

INTERVIEW TARGET

Be able to explain Linux logging, journalctl, logrotate, CPU/memory/disk monitoring, cron scheduling and basic production troubleshooting without referring to notes.

NEXT CHAPTER

Chapter 13: Linux Internals and Kernel Fundamentals

You will learn how the Linux kernel works internally, including process scheduling, virtual memory, file descriptors and system calls, and how to investigate low-level system behavior.

END OF CHAPTER 12